In main() we define a string array a which is printed. Then the replace() function is called with the actual parameters a, the character '/' and the character '-'. The printout will be:

```
C:/Mydocuments/Sheets
C:-Mydocuments-Sheets
```

6.12 Declaration Space

A variable declared inside a function is only valid within the function. The same applies to the formal parameters. In the previous program you can for instance not use the variable s outside the replace() function, like printing s from main().

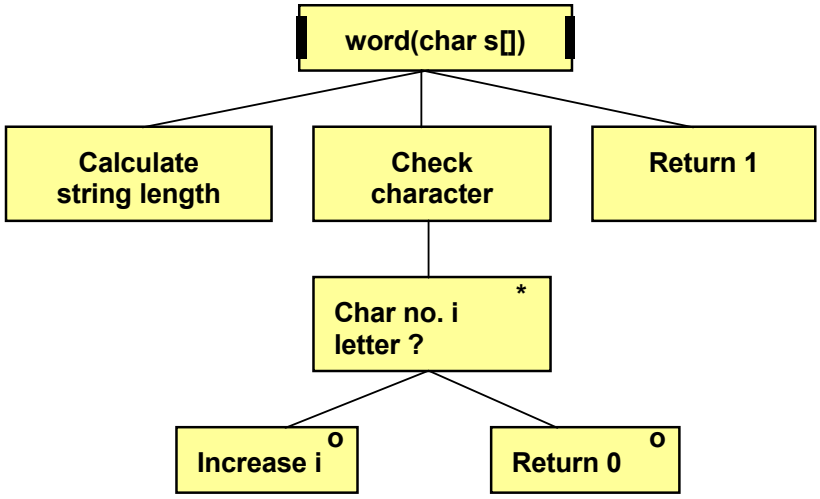
You can neither use a variable in replace() that is declared in main(). For instance, you can't print the variable a from inside of replace().

A variable is valid only in the function where it is declared.

There are however workarounds, but that is beyond the scope of this course.

6.13 The Word Program

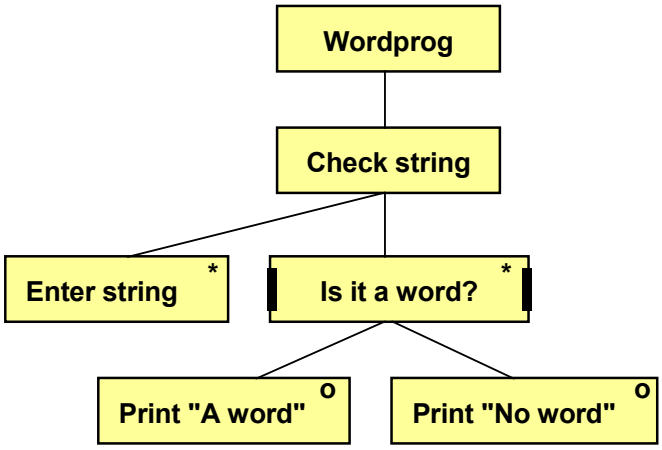
We will now create a function which checks if a string is a word, i.e. only contains the characters a-z or A-Z. The function should be used in a program where the usere repeatedly can enter a word and get it checked. We start with a JSP graph for the function:



The function begins by calculating the string length. The loop 'Check character' goes through character by character and checks if it is in the interval a-z or A-Z. If so, we increase the loop counter i by 1. Otherwise we return the value 0 and the function is terminated. If all characters are letters, the loop will complete and we return 1.

We now give a JSP graph for the program calling the word() function:

Download free eBooks at bookboon.com



The program has a loop 'Check string'. For each turn of the loop we read a string from the keyboard. The string is sent to the function word(). If 1 is returned by the function, we print 'A word'. If 0 is returned, we print 'No word'.

Here is the code:

```
#include <iostream.h>
#include <string.h>
int word (char s[])
{
  int i=0, j;
  j = strlen(s);
  while (i<j)
  {
    char c = s[i];
    if ((c>='a' && c<='z') || (c>='A' && c<='Z'))
      i++;
    else
      return 0;
  }
  return 1;
}
void main()
{
  char str[10];
  while (cin >> str)
    if (word(str))
      cout << "A word" << endl;
    else
      cout << "No word" << endl;
}
```


Download free eBooks at bookboon.com

The function word() begins by calculating the length of the string s and storing it in the variable j.

The while loop has the condition that the loop counter i should be less than the string length j. Character number i is copied to the char variable c.

The if statement checks if c is greater than 'a' and less than 'z' or greater than 'A' and less than 'Z'. If this condition is satisfied, the character is a letter and i is increased by 1. If it is another character, 0 is returned and the function is terminated.

If the while loop is allowed to complete, all characters are letters and the value 1 is returned.

In main() the string array str is declared. The while loop has the condition of a successful string entry, i.e. the user has not pressed Ctrl-Z.

The if statement in main() has a call to word() and sends the entered string as actual parameter. If the function returns 1 (it is a word), the condition is true and the text 'A word' is printed. If 0 is returned, the condition is false and the else statement provides the output 'No word'.

6.14 Override Functions

A function can appear in different shapes. For instance, it can perform a task with different data sets. What differs between the various shapes is the parameter set. If the number of parameters or the data types of the parameters are different, it is considered different function shapes, or override functions. The function header defines the difference. Here are two examples of override functions:

```
void prt(int i, int width)
void prt(char s[])
```

Click on the ad to read more
Download free eBooks at bookboon.com

The functions have the same name (prt), but different parameter sets. The purpose of the functions is to print something on the screen. The first function should print the number i with as many positions as given by the width parameter (using the setw() function). The second function should print the string s.

Here is the code for the two functions:

```
void prt(int i, int width)
{
   cout << setw(width) << i;
}

void prt(char s[])
{
   cout << s;
}
```

The first function uses the number width as parameter to the setw() function, which assigns that number of positions on the screen for the number i.

The second function prints the string sent to the function.

We can now use the function prt() in a program to print a number or a text. Depending on the actual parameters sent to the function, the program will select the appropriate function variant.

For instance, the statement:

```
prt("The number:");
```

will select the second function, while the statements:

```
int k = 8;
prt(k, 3);
```

will select the first function. If we combine these statements:

```
prt("The number:");
int k = 8;
prt(k, 3);
```

we will get the printout:

```
The number:  8
```

with two blanks in front of the 8, since we set width to 3, totally three positions.

Writing override functions in this way provides a flexibility to programming, where the program selects the function variant applicable for the moment.

6.15 Declaration - Definition

We have previously stated that a function should be positioned in front of main() in the program to make the compiler be able to recognize it when called from main(). Here is an example of this:

Download free eBooks at bookboon.com

```
double dAvg(double x1, double x2)
{
   return (x1 + x2)/2;
}
main()
{
   //…
   mv = dAvg(no1, no2);
   //…
}
```

Here, the function dAvg() calculates the average of the two numbers sent to the function. In main() we call dAvg() with the actual parameters no1 and no2, which we assume to have been assigned values previously in the program.

An alternative is to write only the function declaration before main() and let the definition of the function appear after main():

```
double dAvg(double x1, double x2);
main()
{
   //…
   mv = dAvg(no1, no2);
   //…
}
double dAvg(double x1, double x2)
{
   return (x1 + x2)/2;
}
```

The first line declares the function dAvg(). It is exactly identical to the function header, followed by a semicolon. Having declared the function, the compiler knows about it and is recodgnized when called from main().

The definition of the function, i.e. the function header plus the function body, can then be positioned anywhere in the program. If you have several functions declared before main(), you can write the function declarations in any order after main().

This way of first declaring functions and placing the definitions afterwards is common by programmers and provides the advantage of having main() first, which is logical since the execution starts there.

You can declare a function in an abbreviated way:

```
double dAvg(double, double);
```

Here we exclude the formal parameter names and specify only their data types. However, the function header of the function definition must be complete with formal parameter names.

Download free eBooks at bookboon.com

6.16 Header Files

Function declarations are often stored in a separate header file and the function definitions in the corresponding cpp file. The header file must then be included in the program using the functions.

For instance, you can create a cpp file with all your function definitions. Suppose we name it myfunc.cpp. The function declarations are stored in the header file myfunc.h.

When writing the program in still another cpp file, which will call the functions, myfunc.h must be included in the program file.

Here is a set of functions that we have used previously in this chapter:

```
// myfunc.cpp
#include <iostream.h> // Necessary for cout
void underline(int n)
{
   for (int i=1; i<=n; i++)
      cout << "=";
}
double dDiscount(int iQty, double dLinePrice)
{
   if (dLinePrice>1000)
      return 0.15;
```

Click on the ad to read more
Download free eBooks at bookboon.com

```
  else if (dLinePrice>500 || iQty>10)
    return 0.10;
  else
    return 0;
}
double dPrice(int iNo, double dUnitPrice)
{
  const double dTax = 0.25;
  double dLinePr, dDiscPerc;
  dLinePr = iNo * dUnitPrice;
  dDiscPerc = dDiscount(iNo, dLinePr);
  return dLinePr * (1-dDiscPerc)*(1+dTax);
}
```

```
// myfunc.h
void underline(int n);
double dDiscount(int iQty, double dLinePrice);
double dPrice(int iNo, double dUnitPrice);
```

```
// price.cpp
#include <iostream.h>
#include "myfunc.h"
void main()
{
  //…
  cout << "To be paid: " << dPrice(iQty, dUnitPr)<<endl;
  //…
}
```

In the price.cpp program we have included the file myfunc.h. The compiler will look in myfunc.h to ensure that all functions called by the program are declared in myfunc.h.

Note that we include own header files with double quotes instead of the characters < and >. That implies that the compiler looks in different folders to find the header files. The files myfunc.h and myfunc.cpp should be stored in the same folder as price.cpp, while iostream.h is stored in a particular folder created at installation of Visual C++.

Download free eBooks at bookboon.com

6.16.1 Project

When working with several files in this way, you must **create a project** in Visual C++ and add all files to the project. Do as follows:

- Select File - New and indicate the Projects tab.
- Mark the option Win32 Console Application and enter a name of the project in the box Project name. Click OK.
- A window is displayed. Click Finish.
- A confirmation window is displayed. Click OK.

When **creating a new cpp file**, do as follows:

- Select File - New and indicate the Files tab.
- Mark the option C++ Source File and enter a name of the cpp file. Click OK.
- The code window is displayed. Enter your code and click the Save button.
- Add the file to the project by selecting Project - Add To Project - Files. Mark the file and click OK.

When **creating a new header file**, do as follows:

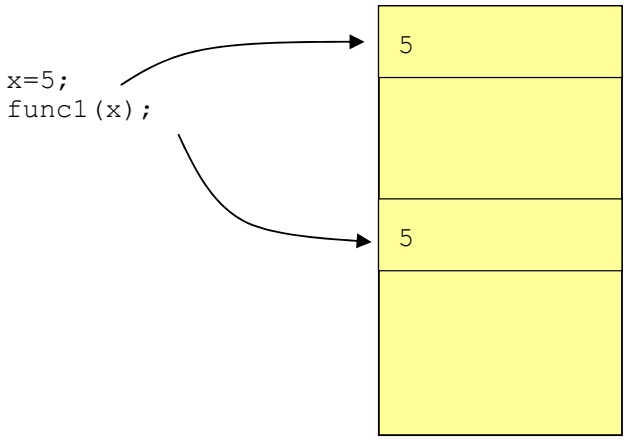
- Select File - New and indicate the Files folder.
- Mark the option C/C++ Header File and enter a name of the header file (the same name as the corresponding cpp file). Click OK.
- The code window is displayed. Enter your code and click the Save button.
- Add the file to the project by selecting Project - Add To Project - Files. Mark the file and click OK.

Compile the entire project by clicking the Build button (the same button as used for compilation so far).

Run the program by clicking the Execute Program button (the same button as used for running a program so far).

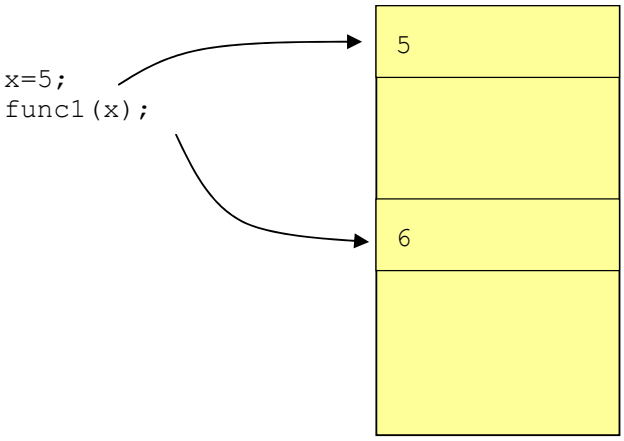
6.17 Reference Parameters

When calling a function with actual parameters, the value for the acutal parameter is copied from its memory area to another memory area used by the function's formal parameter:



Download free eBooks at bookboon.com

The variable x has the value 5 stored in a memory location. When func1() is called, that value is copied to another memory location used by the function. If a statement in the function would change the value to 6, the memory location of the function is affected, but not the original value of x:



Sometimes this is good, but sometimes you also want the original value to be changed.

Another disadvantage is when a lot of data is to be transferred to the function, e.g. at object oriented programming when an object consisting of many Mbyte of data is to be transferred to a function. A lot of memory is then consumed and it takes time to copy huge amounts of data.

The solution is to use a reference parameter. No copying of data is then made, but the original actual parameter and the function's formal parameter point to the same memory location:

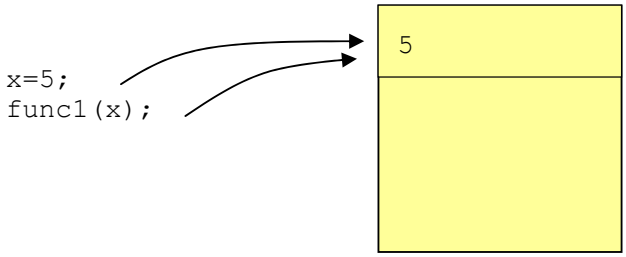
Rand Merchant Bank uses good business to create a better world, which is one of the reasons that the country's top talent chooses to work at RMB. For more information visit us at www.rmb.co.za

Thinking that can change your world

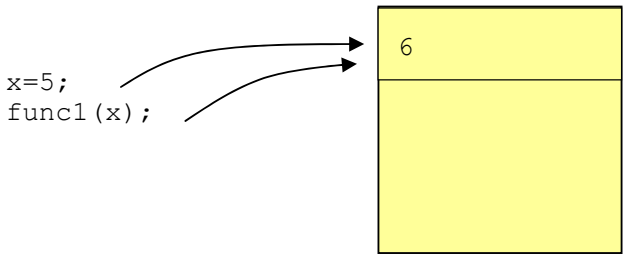
Rand Merchant Bank is an Authorised Financial Services Provider

Click on the ad to read more

Download free eBooks at bookboon.com



If the function func1() changes the value, it also affects the value of the original variable x:



Defining a function parameter as reference parameter is made by placing an & character after the data type:

```
void underline(int& n)
{
   //…
}
```

The parameter n is here a reference parameter.

You call the function in the usual way:

```
underline(iNo);
```

6.18 Parameters with Default Values

Sometimes it would be convenient to exclude an actual parameter when calling a function. The function must then itself be capable of assigning a value to the formal parameter. This is accomplished by defining the formal parameter with a default value, i.e. if no value is supplied by the function call, the formal parameter gets a standard value. Here is an example:

```
void print_many (char c, int iNo=1)
{
   for (int i=1; i<=iNo; i++)
      cout << c;
}
```

The function print_many() prints a character a specified number of times. It takes a character c and an integer iNo as parameters. The character c is printed iNo times.

The for-loop goes from 1 to iNo and prints c for each turn of the loop.

Download free eBooks at bookboon.com

The parameter iNo has the default value 1, which means that if no integer value is sent to the function, iNo will get the value 1. Thus, you specify the default value in the function header:

```
int iNo=1
```

The call to the function can be in two different ways. Here is one:

```
print_many ('x', 4);
```

This call sends the character 'x' and the number 4 to the function. Since we specify a value in the call, the default value for the parameter iNo will be ignored and the value 4 will be used. The output will be:

```
xxxx
```

Here is the other way of calling the function:

```
print_many ('y');
```

Here, we don't send any integer value, so the default value 1 will be used. The character 'y' will be printed once:

```
y
```

One thing you should remember when using a function declaration first and a function definition later, is that the default value for a parameter should only be specified in the declaration of the function, and not be repeated in the function header of the function definition. This is the way it should be written:

```
void print_many (char c, int iNo=1);
void main()
{
   //…
}
void print_many (char c, int iNo)
{
   //…
}
```

Furthermore, the parameter with the default value must be the last one in the parameter list. You can't interchange the parameters c and iNo.

6.19 Recursive Functions

A recursive function is a function that calls itself, i.e. from inside of the function body you call the same function in a program statement. This may sound as an infinite loop. The code must of course be constructed with a condition to make the series of calls be interrupted.

Here is the basic logic for a recursive function:

```
func()
{
   //misc code
```


Download free eBooks at bookboon.com

```
   if (…)
      return func();
   else
      return 1;
}
```

The function func() has a call to itself in the first return statement. This call will be performed as long the if condition is true, repeatedly. But some time the statements before the if statement must imply that the if condition is false. Then the value 1 is returned and the recursive function calls are interrupted.

Recursive functions are mostly used in mathematical applications. We will create a recursive function which calculates the faculty of the number sent to the function.

Some repetition of your math knowledge. Faculty is identified by !. For instance 5! (faculty of 5) = 5 x 4 x 3 x 2 x 1. You start with the number and repeatedly multiply by the number that is 1 less until you arrive at 1.

The function has the following header:

```
int nfac(int n)
```

The function name is nfac and it takes an integer as parameter. We will consequently multiply n by n-1, n-2, n-3 etc. down to 1. We will call nfac() with the number that is 1 less than the number used in the previous call. So the function must have an if statement which checks if the parameter n is = 1. If so, the number 1 should be returned, otherwise the function should be called again, this time with n-1 as actual parameter.

Deloitte.

Discover the truth at www.deloitte.ca/careers

© Deloitte & Touche LLP and affiliated entities.

Click on the ad to read more

Download free eBooks at bookboon.com

Here is the code:

```
int nfac(int n)
{
  if (n<=1)
    return 1;
  else
    return n * nfac(n-1);
}
```

The function has an if statement which checks if the supplied parameter is 1 or less. If so, 1 is returned. Otherwise the product of n and the result of the call to the same function with parameter n-1 is returned.

Suppose we use this call:

```
iFaculty = nfac(4);
```

The number 4 is sent as parameter.
Since 4 is greater than 1, this multiplication is performed: 4 * nfac(3).
The call nfac(3) accordingly gives the multiplication 3 * nfac(2).
The call nfac(2) gives the multiplication 2 * nfac(1).
The call nfac(1) now has the actual parameter 1 and since it provides a true condition in the if statement, 1 is returned, and no more calls are made.

Thus, the resulting multiplication is 4 * 3 * 2 * 1, which is stored in the receiving variable iFaculty.

6.20 Summary

In this chapter we have learnt to write functions. Functions are used in professional programs to split up the code into well-defined sections and to achieve a structure easy to grasp, which facilitates program maintenance.

We have learnt how to send values to, and receive the result from a function. We have also learnt how to use header files in connection with declaration and definition of functions.

We have made a brief introduction to reference parameters – an efficient tool to save memory and improve program performance. You have also seen how to write recursive functions to make the code more efficient.

6.21 Exercises

1. Write a function which calculates and returns the average of three numbers. Call the function from main() and create convenient printouts, so you can check the correctness of the function.
2. Write a function max() which returns the greatest of two numbers. Test the function with a call from main() and complete with suitable printouts.
3. Complete the previous program so that it can calculate the greatest of three numbers by means of the function max().
4. Start from the program 'Least Item of an Array' and complete it with printing of the greatest item of the array by means of the function max().

Download free eBooks at bookboon.com

5. Write three functions which calculate
   - circumference of a rectangle with the sides as parameters
   - area of a rectangle with the sides as parameters
   - price for building a fence around a rectangular field, where the price per meter is 145:- and a gate of 650:-

   Then, write a program that reads the sides of the rectangle from the user and displays the circumference, area and fence price by means of the three functions.
6. Write a function that takes an integer n and prints a list of the squares and cubes of the numbers 1-n. From main(), read the number n from the user.
7. Write a function printLine(int n, char c) which prints the character c at a line as many times as specified by the integer n. Use the function in a program which prints a frame consisting of asterixes (*) on the screen.
8. Write a function expandWord(char cWord[]) which prints the text in the parameter 'cWord' with one blank between each character. E.g. the word `Data` is printed as `D a t a.` Also write a main() program which reads a text from the user and calls the function.
9. Write the following functions:
   - void initial(char str[]) which prints the initials of the name str.
   - void revers(char str[]) which interchanges the first and surname of str and prints it.
   - int lgth(char str[]) which returns the length of str.
   - void back(char str[]) which prints the name backwards.
   - void upper(char str[]) which prints the name in upper case.

   Use the functions in a main() program.
10. Start from the program with the price calculation in the section 'Function and Subfunction'. Write one more subfunction which is used by the function dPrice() and which reads a customer category from the user and returns still another discount percent (A=5%, B=7%, C=9%). Ensure that the function takes erroneous entry into account. Modify the function dPrice() to provide a correct price calculation. Save this program, we will use it in later exercises.
11. Write a program for calculation of car rent. The program should contain a function that calculates the daily charge (500:-) plus kilometer charge (1:40 per km) plus the fuel price. The calculation of the kilometerage is made by a subfunction which prompts the user for start and end odometer value and returns the number of kilometers driven. The fuel price is calculated by another subfunction which reads the fuel consumption and returns the fuel charge (9.27 per litre). Save this program, we will use it in later exercises.
12. Suppose you want to create a function that prints the letters å, ä and ö correctly. The function takes a string as parameter, searches for the letter combinations aa, ae and oe, end replaces them by å, ä and ö respectively. In main() you read a string from the user and call the function. To make it work, the user must enter the word 'båda' as 'baada'.
13. Start from the 'Word program' earlier in this chapter. Change it to a digit program, i.e. the program should check an entered string to only contain digits and decimal point.
14. Write a "playing card" function which takes a card value (1-13) and prints the correct value (2-10, 'jack', 'queen', 'king', 'Ace'). Use the function in a main() program.
15. Improve the previous function to also take a colour parameter (1-4) and prints 'hearts', 'clubs', 'diamonds' or 'spades'.

Download free eBooks at bookboon.com

16. Write the following boolean functions:
    - bool odd(int n) which gives the value true if n is odd.
    - bool divable(int a, int b) which returns true if a is evenly dividable by b.
    - bool digit(char c[]) which returns true if the first character of c is a digit.
    - bool letter(char c[]) which returns true if the first character of c is a letter.

    Use the functions in if statements which print the results.
17. Write a program which works as a calculator. There should be one main() function which reads the calculation, for instance 5 * 3, and four functions, one for each type of calculation + - * /.
18. Write a program that calculates the average score for a student. The program should prompt the user for course scores (MVG=20, VG=15, G=10, IG=0) and number of hours that the course comprises. These two values are multiplied. The entry goes on until all courses are complete. The course scores of all courses are added. The sum is divided by the total number of hours for all courses, which gives the average score. The transfer between score (for instance VG) to value (15) is made by a function.
19. Change the price calculation program in exercise 10 above, so that you get two override functions for discount calculation with the same function names. One of them takes the total price and quantity as parameters and the other takes a customer group as parameter. The entry of customer group from the user must then be made in the dPrice() function.
20. Modify the previous program so that you place the function declarations first and the function definitions after main().
21. Start from exercise 11 and put all code in a project with the function definitions in a separate cpp file, the function declarations in corresponding header file and main() in a separate cpp file.
22. Modify the previous exercise so that the parameters of the functions are used as reference parameters.
23. Change the function for calculation of the fuel price in the previous exercise so that it takes the litre price as parameter. It should have the default value 9.32.
24. Write a function which creates a number of random rolls of a dice. The number of rolls should be taken as a parameter with the default value 5. The function should return the average of the rolls. Use this function in a program where the user can enter a number of rolls and get the average printed. Store the program in a project with separate cpp files for the function and main() and a header file with the function declaration.
25. Expand the previous exercise so that the program runs the function using the default value, i.e. without sending the number of rolls to the function.
26. Improve the previous exercise so that you play against the computer and the program informs you about who won. The comparison between your and the program's score should be made by a function.
27. Change the previous exercise so that the function uses reference parameters.
28. Start from the function nfac() in the 'Recursive Functions' section and place it in a program where the user can enter the integer and get the faculty of it printed.
29. Write a recursive function which sums the integers n, n-1, n-2,… 1. Use it in a main() program.
30. Change the previous function so that it every other time adds and subtracts, for instance 6-5+4-3+2-1.

Download free eBooks at bookboon.com